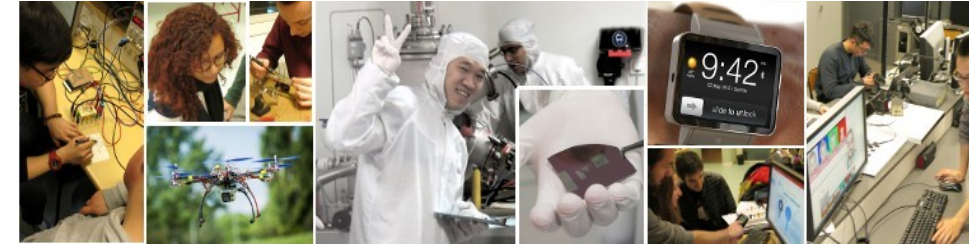




POLYTECH[®]
ORLÉANS

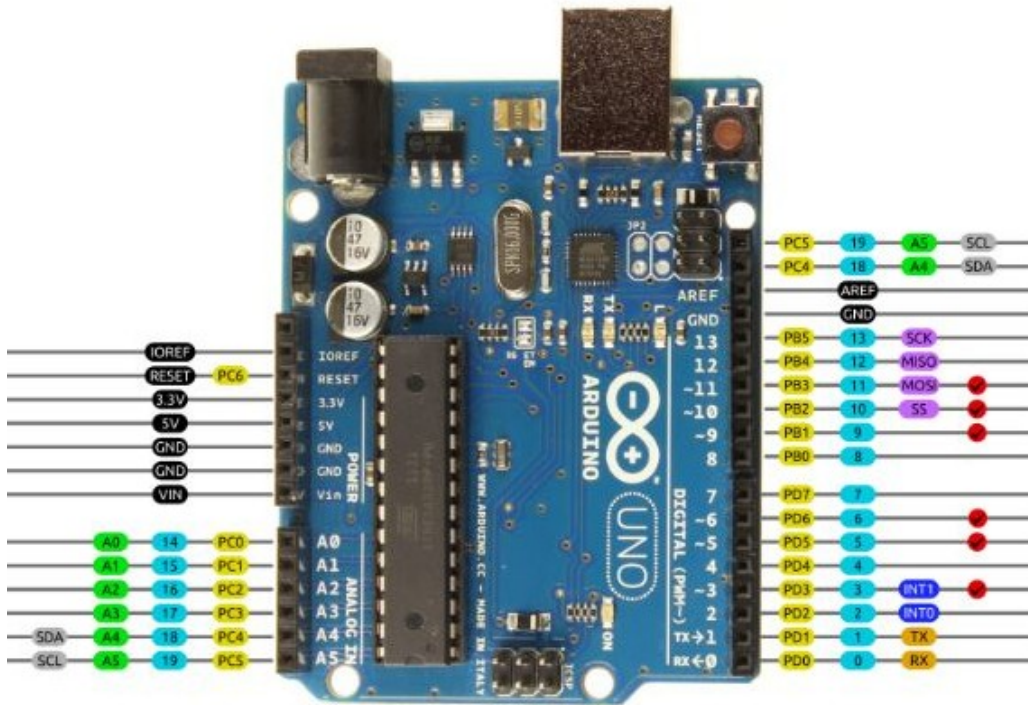
École d'Ingénieurs de l'Université d'Orléans



Interruptions

(Ro. Weber)

Back to the Processor architecture



Add a button connected to a led :

- 1) When I push the button, the led is on and a counter is incremented (to be displayed on the serial monitor). Otherwise nothing happens.
- 2) The build-in led must blink with a period of approximatively 2 s.

use <https://www.tinkercad.com>

AVR DIGITAL ANALOG POWER SERIAL SPI I2C PWM INTERRUPT

Back to the Processor architecture

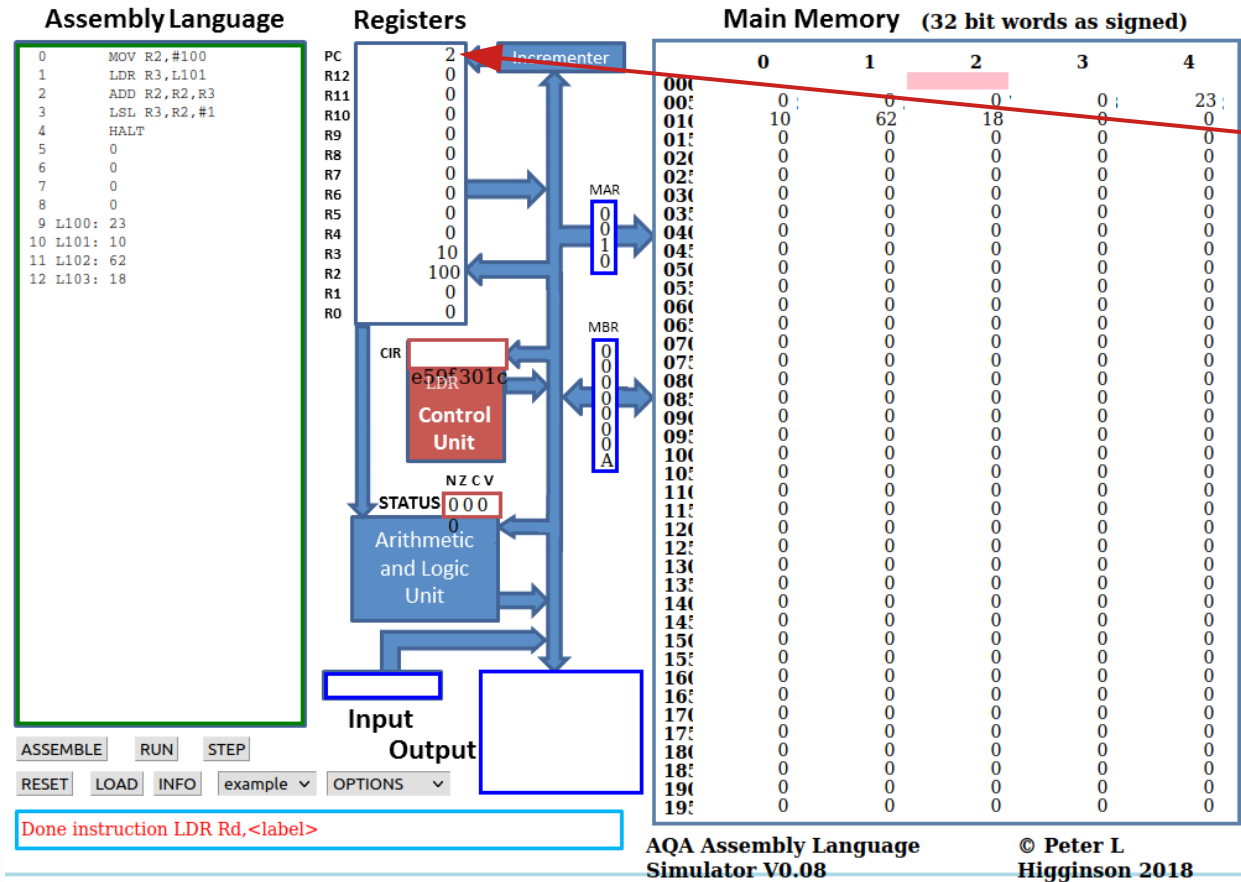
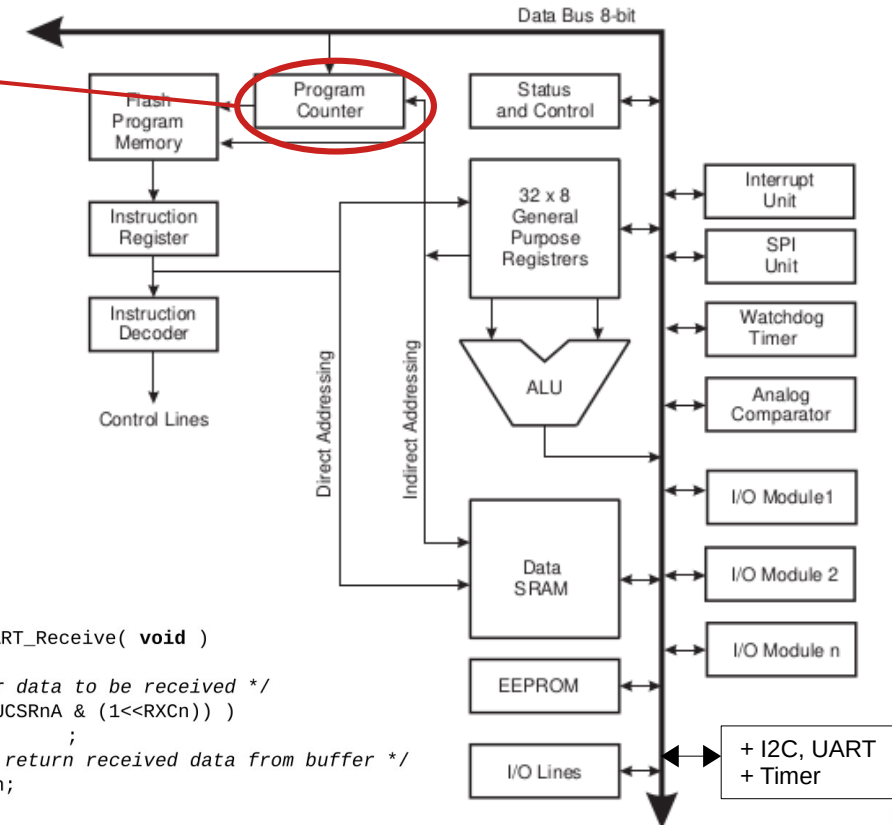


Figure 7-1. Block Diagram of the AVR Architecture

Arduino architecture (from datasheet)



```

unsigned char USART_Receive( void )
{
    /* Wait for data to be received */
    while ( !(UCSRnA & (1<<RXn)) )
        ;
    /* Get and return received data from buffer */
    return UDRn;
}
    
```

<http://www.peterhigginson.co.uk/AQA/>

To interact with peripherals :

- polling (you check the peripheral status periodically by yourself)
- interruption (the peripheral triggers an action by itself)

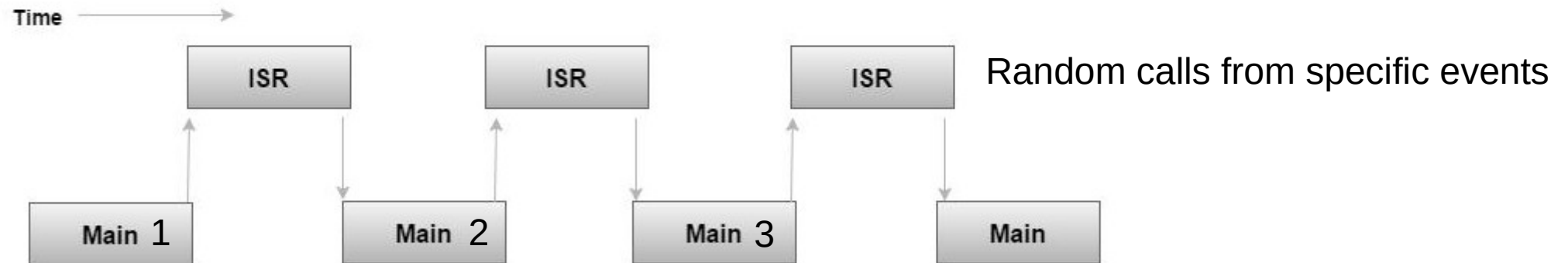


■ Interruption principles

Program Execution without Interrupts



Program Execution with Interrupts



ISR : Interrupt Service Routine



Interruption principles (ATMEGA example)



Look at the data sheet (page 65) of the arduino processor

Interruption name

Table 12-6. Reset and Interrupt Vectors in ATmega328 and ATmega328P

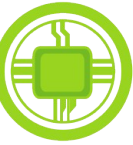
VectorNo.	Program Address ⁽¹⁾	Source	Interrupt Definition
1	0x0000 ⁽¹⁾	RESET	External Pin, Power-on Reset, Brown-out Reset and Watchdog System Reset
2	0x0002	INT0	External Interrupt Request 0
3	0x0004	INT1	External Interrupt Request 1
4	0x0006	PCINT0	Pin Change Interrupt Request 0
5	0x0008	PCINT1	Pin Change Interrupt Request 1
6	0x000A	PCINT2	Pin Change Interrupt Request 2
7	0x000C	WDT	Watchdog Time-out Interrupt
8	0x000E	TIMER2 COMPA	Timer/Counter2 Compare Match A
9	0x0010	TIMER2 COMPB	Timer/Counter2 Compare Match B
10	0x0012	TIMER2 OVF	Timer/Counter2 Overflow
11	0x0014	TIMER1 CAPT	Timer/Counter1 Capture Event
12	0x0016	TIMER1 COMPA	Timer/Counter1 Compare Match A
13	0x0018	TIMER1 COMPB	Timer/Counter1 Compare Match B
14	0x001A	TIMER1 OVF	Timer/Counter1 Overflow
15	0x001C	TIMER0 COMPA	Timer/Counter0 Compare Match A
16	0x001E	TIMER0 COMPB	Timer/Counter0 Compare Match B
17	0x0020	TIMER0 OVF	Timer/Counter0 Overflow
18	0x0022	SPI, STC	SPI Serial Transfer Complete
19	0x0024	USART, RX	USART Rx Complete
20	0x0026	USART, UDRE	USART, Data Register Empty
21	0x0028	USART, TX	USART, Tx Complete
22	0x002A	ADC	ADC Conversion Complete
23	0x002C	EE READY	EEPROM Ready
24	0x002E	ANALOG COMP	Analog Comparator
25	0x0030	TWI	2-wire Serial Interface
26	0x0032	SPM READY	Store Program Memory Ready

Address	Labels	Code	Comments
0x0000		jmp	RESET ; Reset Handler
0x0002		jmp	EXT_INT0 ; IRQ0 Handler
0x0004		jmp	EXT_INT1 ; IRQ1 Handler
0x0006		jmp	PCINT0 ; PCINT0 Handler
0x0008		jmp	PCINT1 ; PCINT1 Handler
0x000A		jmp	PCINT2 ; PCINT2 Handler
0x000C		jmp	WDT ; Watchdog Timer Handler
0x000E		jmp	TIM2_COMPA ; Timer2 Compare A Handler
0x0010		jmp	TIM2_COMPB ; Timer2 Compare B Handler
0x0012		jmp	TIM2_OVF ; Timer2 Overflow Handler
0x0014		jmp	TIM1_CAPT ; Timer1 Capture Handler
0x0016		jmp	TIM1_COMPA ; Timer1 Compare A Handler
0x0018		jmp	TIM1_COMPB ; Timer1 Compare B Handler
0x001A		jmp	TIM1_OVF ; Timer1 Overflow Handler
0x001C		jmp	TIM0_COMPA ; Timer0 Compare A Handler
0x001E		jmp	TIM0_COMPB ; Timer0 Compare B Handler
0x0020		jmp	TIM0_OVF ; Timer0 Overflow Handler
0x0022		jmp	SPI_STC ; SPI Transfer Complete Handler
0x0024		jmp	USART_RXC ; USART, RX Complete Handler
0x0026		jmp	USART_UDRE ; USART, UDR Empty Handler
0x0028		jmp	USART_TXC ; USART, TX Complete Handler
0x002A		jmp	ADC ; ADC Conversion Complete Handler
0x002C		jmp	EE_RDY ; EEPROM Ready Handler
0x002E		jmp	ANA_COMP ; Analog Comparator Handler
0x0030		jmp	TWI ; 2-wire Serial Interface Handler
0x0032		jmp	SPM_RDY ; Store Program Memory Ready Handler
0x0034	RESET:	ldi	r16, high(RAMEND); Main program start
0x0035		out	SPH,r16 ; Set Stack Pointer to top of RAM
0x0036		ldi	r16, low(RAMEND)
0x0037		out	SPL,r16
0x0038		sei	; Enable interrupts
0x0039		<instr>	xxx

Priority order

Assuming that the interruption has been enabled, when the corresponding events occurs:

- 1) the processor finished the current instruction
- 2) the register context is saved in the stack, in particular the Pc register
- 3) the PC reg (Program Counter) is loaded with the interrupt vector/address
- 4) the corresponding instruction is executed: Usually a jump to the interruption subroutine (written by you)
- 5) at the end of this subroutine, the data stored in the stack are reloaded in their corresponding registers. In particular the PC point back to the next main program instruction to be executed



High level configuration (arduino example)

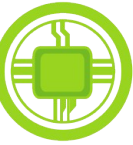
(<https://www.arduino.cc/reference/en/language/functions/external-interrupts/attachinterrupt>)

Work only for external interruptions

By using the arduino library, Try to implement the same experiment with the external interruption int0



Other example : sleep mode (eco-conception)



```
#include <avr/sleep.h>

// interrupt service routine in sleep mode
void wake ()
{
    sleep_disable ();           // first thing after waking from sleep:
    detachInterrupt (digitalPinToInterrupt (2));    // stop LOW interrupt on D2
} // end of wake

void sleepNow ()
{
    set_sleep_mode (SLEEP_MODE_PWR_DOWN);
    noInterrupts ();           // make sure we don't get interrupted before we sleep
    sleep_enable ();           // enables the sleep bit in the mcucr register
    attachInterrupt (digitalPinToInterrupt (2), wake, LOW); // wake up on low level on D2
    interrupts ();             // interrupts allowed now, next instruction WILL be executed
    sleep_cpu ();              // here the device is put to sleep
} // end of sleepNow
```



How to enable an interruption : low level approach (ATMEGA example)



- 1) As usual, by configuring specific registers :
 - a) General purpose (see SREG page 11)
 - b) specific registers linked to the interruption →
- 2) writing a subroutine also called :
Interrupt Service Routine (ISR)
the name of your subroutine must be the same as the one used in the interrupt table.
- 3) configuring the interrupt table with the address (compiler job) of your subroutine with the macro `ISR( your_isr_name)`
In particular the ISR() will add a prologue and an epilogue (push/pull register context in/from the stack, `RETI()`...}

See

https://www.nongnu.org/avr-libc/user-manual/group__avr__interrupts.html

Example the external interruptions (page 70)

Table 12-6. Reset and Interrupt Vectors in ATmega328 and ATmega328P

VectorNo.	Program Address ⁽²⁾	Source	Interrupt Definition
1	0x0000 ⁽¹⁾	RESET	External Pin, Power-on Reset, Brown-out Reset and Watchdog System Reset
2	0x0002	INT0	External Interrupt Request 0
3	0x0004	INT1	External Interrupt Request 1
4	0x0006	PCINT0	Pin Change Interrupt Request 0
5	0x0008	PCINT1	Pin Change Interrupt Request 1
6	0x000A	PCINT2	Pin Change Interrupt Request 2
7	0x000C	WDT	Watchdog Time-out Interrupt

Example the SPI interruptions (page 160)

```
0x001E    jmp    TIM0_COMPB ; Timer0 Compare B Handler
0x0020    jmp    TIM0_OVF  ; Timer0 Overflow Handler
0x0022    jmp    SPI_STC   ; SPI Transfer Complete Handler
0x0024    jmp    USART_RXC ; USART, RX Complete Handler
0x0026    jmp    USART_UDRE ; USART, UDR Empty Handler
0x0028    jmp    USART_TXC ; USART, TX Complete Handler
```

```
volatile char buf [100];
volatile byte pos;

// SPI interrupt routine
ISR (SPI_STC_vect)
{
    byte c = SPDR; // grab byte from SPI Data Register

    // add to buffer if room
    if (pos < sizeof buf)
    {
        buf [pos++] = c;
    } // end of room available
} // end of interrupt routine SPI_STC_vect
```



Tips and tricks

When writing an Interrupt Service Routine (ISR):

- Keep it short —————> The most that an interrupt (service) routine might do would be to remember that something append. This fact would then be tested in the main program loop.
- Don't use delay ()
- Don't do serial prints
- Make variables shared with the main code volatile
- Better optimization needs sometimes to write your own full ISR (l.e prologue/epilogue) : atmega example with ISR_NAKED attribute
- For debug, it could be interesting to design BADISR_vect ISR (to catch misuse of interruption vectors)

```
volatile boolean flag; // Force the compiler to not optimize these variables (i.e. not to use the stack or a register)

// Interrupt Service Routine (ISR)
void isr ()
{
    flag = true;
} // end of isr

void setup ()
{
    attachInterrupt (digitalPinToInterrupt (2), isr, CHANGE); // attach interrupt handler
} // end of setup

void loop ()
{
    if (flag)
    {
        // interrupt has occurred
    }
} // end of loop
```

